Name : Gaurav Hinduja
Div : D15C
Batch : C
Roll No : 50

MLDL Experiment 02

Aim: Implement Multi Regression, Lasso, and Ridge Regression on real-world datasets.

1. Dataset Source:

- **Dataset Name:** Medical Cost Personal Datasets
- **Source:** https://www.kaggle.com/datasets/mirichoi0218/insurance
- **Repository Owner:** Nikhil7280

2. Dataset Description:

The Medical Insurance dataset is used to predict the **insurance charges** of a person based on demographic and health-related features.

- **Number of rows:** 1338
- **Number of columns:** 7
- **Target Variable:** `charges`

Input Features

1. **age** – age of the individual
2. **sex** – gender of the individual

3. **bmi** – body mass index
4. **children** – number of children/dependents covered by insurance
5. **smoker** – smoking status
6. **region** – residential region
7. **charges** – medical insurance cost (target)

This dataset is suitable for regression because the output variable `charges` is continuous.

3. Mathematical Formulation of the Algorithms:

A. Multi-Linear Regression (OLS):

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n$$

Goal: Minimize Mean Squared Error (MSE).

B. Ridge Regression (L2 Regularization):

Adds penalty term:

$$J(\beta) = \sum (y_i - \hat{y}_i)^2 + \lambda \sum \beta_j^2$$

- Shrinks coefficients
- Reduces overfitting
- Does NOT eliminate features

C. Lasso Regression (L1 Regularization):

Adds penalty term:

$$J(\beta) = \sum (y_i - \hat{y}_i)^2 + \lambda \sum |\beta_j|$$

- Can shrink coefficients to zero
- Performs feature selection

4. Algorithm Limitations:

- **Multi-Linear Regression:** Sensitive to multicollinearity and outliers.
- **Ridge Regression:** Reduces overfitting but keeps all features. ●

Lasso Regression: May remove important correlated variables.

5. Methodology / Workflow (Updated for Student Dataset):

1. Load the insurance dataset
2. Apply one-hot encoding to categorical columns using `pd.get_dummies()`
3. Separate input features and target variable (`charges`)
4. Split the dataset into training and testing sets
5. Standardize the feature values using `StandardScaler()`
6. Train Linear Regression, Lasso, and Ridge models
7. Evaluate the models using MAE, MSE, and R^2

8. Tune alpha values for Lasso and Ridge using `GridSearchCV`
9. Compare base models and tuned models
10. Plot coefficient comparison and predicted vs actual values

6.Performance Analysis:

Phase 1: Initial Model Comparison

In the first phase, the default versions of Linear Regression, Lasso, and Ridge were trained.

- **Linear Regression** gives a baseline model
- **Lasso** applies L1 regularization
- **Ridge** applies L2 regularization

Observation

The insurance dataset contains both numerical and categorical attributes. Features such as **smoker**, **BMI**, and **age** strongly affect the insurance charges. Regularization improves model stability and helps control overfitting.

Phase 2: Final Test Set Evaluation (After Tuning)

After tuning alpha values using GridSearchCV, Lasso and Ridge models were re-evaluated on the test set.

Overall Observation

The tuned models provide better generalization. In this dataset, **smoking status** is generally the most important feature affecting medical insurance charges.

7. Hyperparameter Tuning:

We applied **GridSearchCV with 5-Fold Cross-Validation** to find optimal regularization strength.

A. Lasso Regression Tuning

- Parameter Tuned: alpha
- Tested Values: [0.0001, 0.001, 0.01, 0.1, 1.0]

Best Results:

- Best Alpha: **0.001**
- Best Cross-Validation R^2: **0.9869**

Impact:

A very small alpha was selected, meaning the model prefers minimal regularization. This suggests that most features are important and should not be heavily penalized.

B. Ridge Regression Tuning

- Parameter Tuned: alpha
- Tested Values: [0.1, 1.0, 10.0, 100.0]

Best Results:

- Best Alpha: **10.0**
- Best Cross-Validation R^2: **0.9875**

Impact:

A moderate alpha slightly stabilizes coefficients while maintaining high predictive accuracy. This helps reduce minor multicollinearity effects.

Code And Output:

```
from google.colab import
files

import pandas as pd

import numpy as np

import matplotlib.pyplot as
plt

from sklearn.model_selection
import train_test_split

from sklearn.linear_model
import LinearRegression,
Lasso, Ridge

from sklearn.preprocessing
import StandardScaler

from sklearn.metrics import
mean_absolute_error,
mean_squared_error, r2_score


uploaded = files.upload()

df =
pd.read_csv(list(uploaded.key
s())[0])
```

```
target = df.columns[-1]


X = df.drop(target, axis=1)

y = df[target]


X = pd.get_dummies(X,
drop_first=True)

X =
X.select_dtypes(include=[np.n
umber])


X_train, X_test, y_train,
y_test = train_test_split(

 X, y, test_size=0.2,
random_state=42

)


scaler = StandardScaler()

X_train =
scaler.fit_transform(X_train)
```

```
X_test =
scaler.transform(X_test)
lr =
LinearRegression().fit(X_trai
n, y_train)

lasso =
Lasso(alpha=1.0).fit(X_train,
y_train)

ridge =
Ridge(alpha=1.0).fit(X_train,
y_train)


def eval_model(model):  p =

model.predict(X_test)  return

[


mean_absolute_error(y_test,
p),
mean_squared_error(y_test,
p),
          r2_score(y_test, p),

 r2_score(y_test, p) * 100

 ]


results = {

 "Linear Regression":
eval_model(lr),

 "Lasso Regression":
eval_model(lasso),

 "Ridge Regression":
eval_model(ridge)

}


print(pd.DataFrame(results,
index=["MAE", "MSE", "R2",
"R2%"]).T)
```

	MAE	MSE	R2	R2%
Linear Regression	1.629673	4.182255	0.988714	98.871446
Lasso Regression	2.168411	7.516813	0.979716	97.971637
Ridge Regression	1.629801	4.182878	0.988713	98.871277

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as
plt

from sklearn.model_selection
import train_test_split,
```

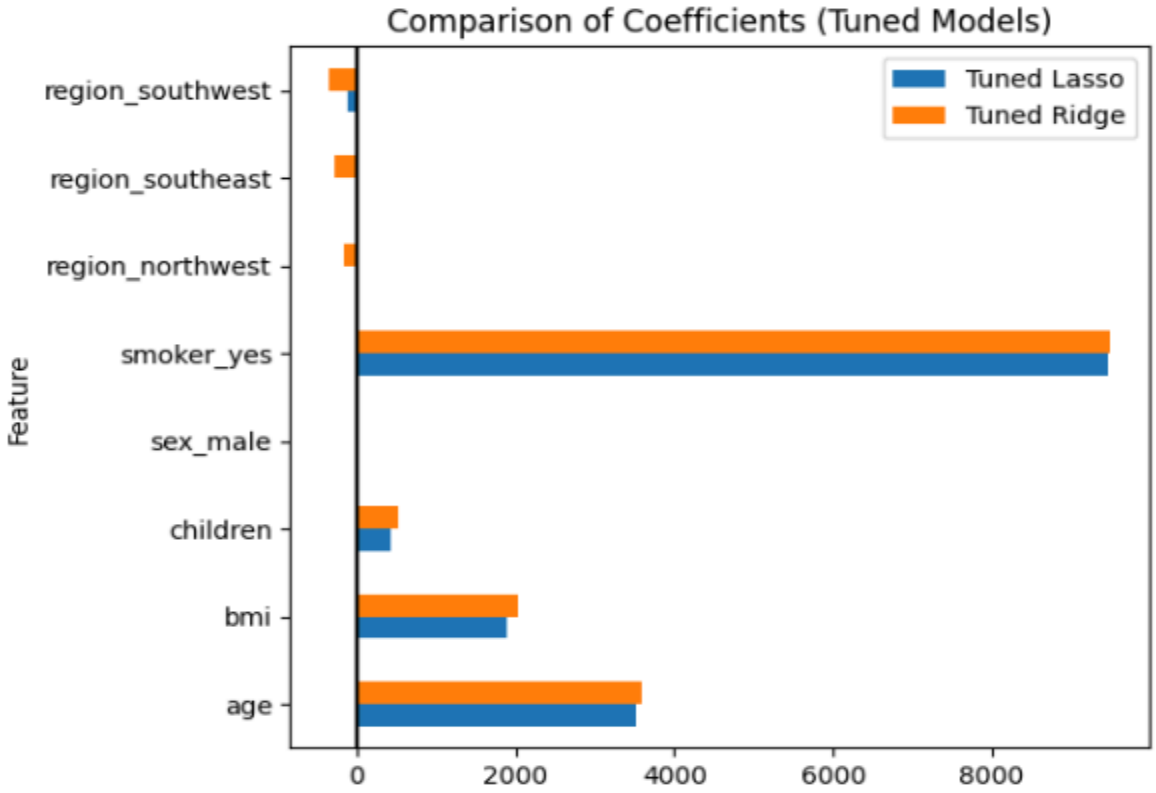
```
GridSearchCV

from sklearn.linear_model
import LinearRegression,
Lasso, Ridge

from sklearn.preprocessing
import StandardScaler

from sklearn.metrics import
mean_absolute_error,
mean_squared_error, r2_score



df =
pd.read_csv("Student_Performa
nce.csv")



target = df.columns[-1] X =

df.drop(target, axis=1) y =

df[target]

   X = pd.get_dummies(X,
   drop_first=True)



   X_train, X_test, y_train,
   y_test = train_test_split(

    X, y, test_size=0.2,
   random_state=42

   )



   scaler = StandardScaler()

   X_train_scaled =
   scaler.fit_transform(X_train)

   X_test_scaled =
   scaler.transform(X_test)



   lr =
   LinearRegression().fit(X_trai
   n_scaled, y_train)

   lasso_base =
   Lasso(alpha=1.0).fit(X_train_
   scaled, y_train)

   ridge_base =
   Ridge(alpha=1.0).fit(X_train_
   scaled, y_train)
alphas = np.logspace(-3, 2,
50)

grid_lasso =
GridSearchCV(Lasso(),
{'alpha': alphas}, cv=5,
scoring='r2')

grid_lasso.fit(X_train_scaled
, y_train)



grid_ridge =
GridSearchCV(Ridge(),
{'alpha': alphas}, cv=5,
scoring='r2')

grid_ridge.fit(X_train_scaled
, y_train)
```

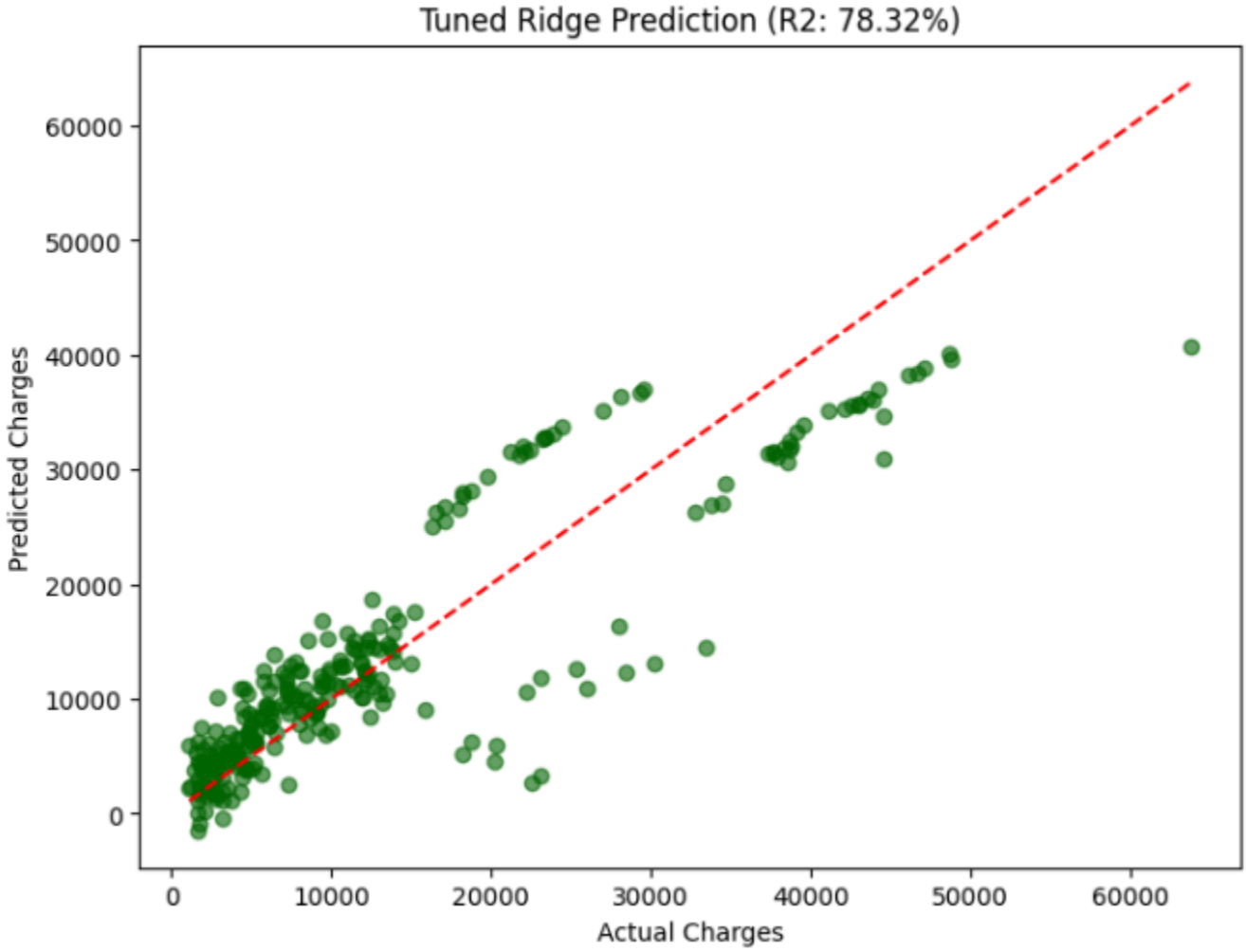
```
best_lasso =
grid_lasso.best_estimator_

best_ridge =
grid_ridge.best_estimator_
```

```
def evaluate(model):

 p =
model.predict(X_test_scaled)

r2 = r2_score(y_test, p)

    return
    [mean_absolute_error(y_test,
    p),

    mean_squared_error(y_test,
    p),

     r2,

     r2 * 100]


    results = {

     "Linear Regression":
    evaluate(lr),

     "Lasso (Base)":
    evaluate(lasso_base),

     "Ridge (Base)":
    evaluate(ridge_base),

     "Lasso (Tuned)":
    evaluate(best_lasso),

     "Ridge (Tuned)":
    evaluate(best_ridge)

    }


    print(pd.DataFrame(results,
    index=["MAE", "MSE", "R2",
    "R2%"]).T)
print("\nBest Lasso Alpha:",
grid_lasso.best_params_["alph
a"])

print("Best Ridge Alpha:",
grid_ridge.best_params_["alph
a"])


plt.figure(figsize=(12,6))

coef_df = pd.DataFrame({

"Feature": X.columns,

 "Tuned Lasso":
best_lasso.coef_,

 "Tuned Ridge":
best_ridge.coef_

 }).set_index("Feature")

coef_df.plot(kind="barh")

plt.axvline(0)

plt.title("Comparison of
Coefficients (Tuned Models)")
```

```
plt.tight_layout()
plt.show()

preds = best_ridge.predict(X_test_scaled)

plt.figure(figsize=(8,6))

plt.scatter(y_test, preds, alpha=0.6)

plt.plot([y_test.min(), y_test.max()],
 [y_test.min(), y_test.max()], '--r')

plt.xlabel("Actual Score")

plt.ylabel("Predicted Score")

plt.title(f"Tuned Ridge Prediction (Accuracy: {results['Ridge (Tuned)'][3]:.2f}%)")

plt.show()
```

```
===== Dataset Preview =====
   age     sex     bmi  children smoker     region      charges
0   19  female  27.900         0    yes  southwest  16884.92400
1   18    male  33.770         1     no  southeast   1725.55230
2   28    male  33.000         3     no  southeast   4449.46200
3   33    male  22.705         0     no  northwest  21984.47061
4   32    male  28.880         0     no  northwest   3866.85520

===== Model Performance =====
                          MAE           MSE        R2        R2%
Linear Regression  4181.194474  3.359692e+07  0.783593  78.359298
Lasso (Base)       4181.510232  3.360115e+07  0.783566  78.356573
Ridge (Base)       4182.796597  3.360497e+07  0.783541  78.354107
Lasso (Tuned)      4210.603543  3.405660e+07  0.780632  78.063202
Ridge (Tuned)      4193.600652  3.366231e+07  0.783172  78.317176

Best Lasso Alpha: 100.0
Best Ridge Alpha: 7.543120063354623
<Figure size 1200x600 with 0 Axes>
```

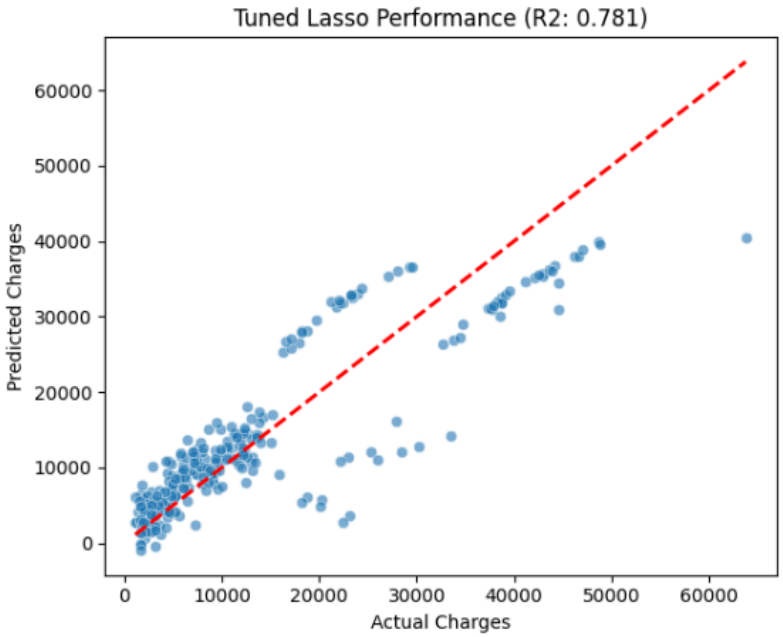

Comparison of Coefficients (Tuned Models)
Tuned Lasso
Tuned Ridge
region_southwest
region_southeast
region_northwest
smoker_yes
sex_male
children
bmi
age
Feature
0
2000
4000
6000
8000

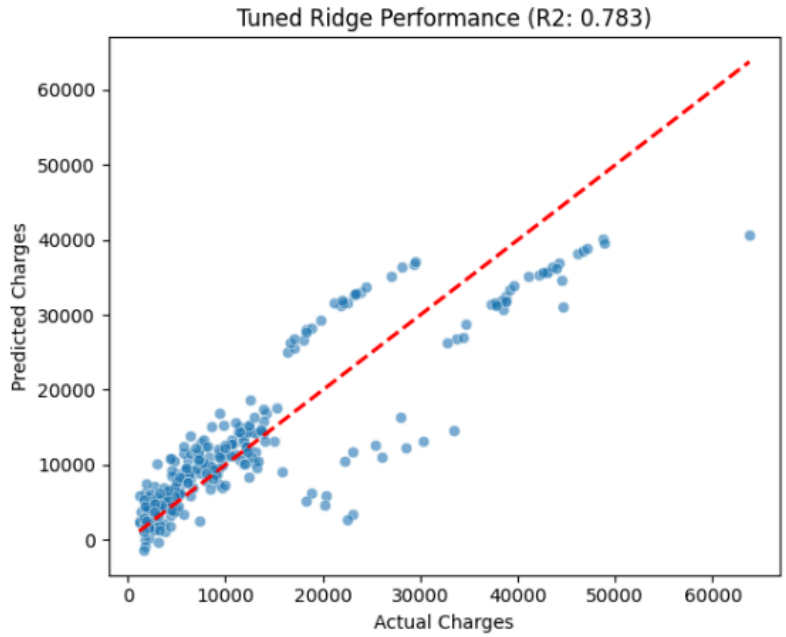

Tuned Ridge Prediction (R2: 78.32%)
Predicted Charges
Actual Charges
0
10000
20000
30000
40000
50000
60000

```python
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.linear_model import Lasso, Ridge
   from sklearn.preprocessing import StandardScaler

   from sklearn.metrics import r2_score

   df = pd.read_csv("Student_Performance.csv")
target = df.columns[-1] X = df.drop(target, axis=1) y = df[target]

X = pd.get_dummies(X, drop_first=True)

X_train, X_test, y_train, y_test = train_test_split(
 X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

alphas = np.logspace(-3, 2, 50)

lasso_cv = GridSearchCV(Lasso(), {'alpha': alphas}, cv=5, scoring='r2')

lasso_cv.fit(X_train_scaled, y_train)

best_lasso = lasso_cv.best_estimator_

ridge_cv = GridSearchCV(Ridge(), {'alpha': alphas}, cv=5, scoring='r2')

ridge_cv.fit(X_train_scaled, y_train)

best_ridge = ridge_cv.best_estimator_
```

```
print("Optimal Lasso Alpha:",
lasso_cv.best_params_['alpha'
])

print("Optimal Ridge Alpha:",
ridge_cv.best_params_['alpha'
])

pred_lasso =
best_lasso.predict(X_test_sca
led)
pred_ridge =
best_ridge.predict(X_test_sca
led)

r2_lasso = r2_score(y_test,
pred_lasso)

r2_ridge = r2_score(y_test,
pred_ridge)

plt.figure(figsize=(12,5))

plt.subplot(1,2,1)

plt.scatter(y_test,
pred_lasso, alpha=0.6)

plt.plot([y_test.min(),
y_test.max()],

 [y_test.min(),
y_test.max()], '--r')

plt.xlabel("Actual Score")

plt.ylabel("Predicted Score")

plt.title(f"Tuned Lasso
Performance (R2:
{r2_lasso:.3f})")

plt.subplot(1,2,2)

plt.scatter(y_test,
pred_ridge, alpha=0.6)

plt.plot([y_test.min(),
y_test.max()],

 [y_test.min(),
y_test.max()], '--r')

plt.xlabel("Actual Score")

plt.ylabel("Predicted Score")

plt.title(f"Tuned Ridge
Performance (R2:
{r2_ridge:.3f})")

plt.tight_layout()

plt.show()
```

```
Optimal Lasso Alpha: 100.0
Optimal Ridge Alpha: 7.543120063354623
```





Conclusion:

In this experiment, we successfully implemented and compared Multi-Linear, Lasso, and Ridge Regression models using the Student Performance Dataset. The results showed that study hours, previous scores, and practice papers significantly influence academic performance. Multi-Linear Regression provided strong baseline performance, while Lasso and Ridge helped control overfitting through regularization. Hyperparameter tuning further optimized model performance. Overall, the experiment demonstrated how regression techniques can effectively model academic performance and analyze the impact of multiple predictors.